Workshop 8: Machine learning in R

Summary sheet

tidymodels

tidymodels is a collection of R packages for **modelling** and **machine learning** that share common step-by-step processes.

Modelling: Supervised ML

Supervised ML models are trained on **labelled data**, learning to map inputs to known outputs.

❶ Create training and testing data



Train on one dataset & test on another to evaluate real-world performance

```
df_split <- initial_split(
     df, prop = 0.75,
     strata = group)
df_train <- training(df_split)
df_test <- testing(df_split)
```

❷ Define pre-processing recipe



Specify the variables to include in the model, using a statistical formula (with ~)

use ~. to include all columns

```
my_recipe <- recipe(df ~ .,
     data = df_train) %>%
     step_normalise(
     all_numeric_predictors())
```

add a normalisation pre-processing step

❸ Specify model, engine and mode



E.g. to use a **random forest** model from the ranger **engine** (package) in **classifer** mode:

```
rf_model <-
     rand_forest(trees = 500) %>%
     set_engine("ranger") %>%
     set_mode("classification")
```

❹ Create workflow



Add your pre-processing **recipe** (step 2) and **model** (step 3) to create a **workflow**

```
rf_wkfl <- workflow() %>%
     add_recipe(my_recipe) %>%
     add_model(rf_model)
```

❺ Fit workflow and get predictions



Use your workflow to fit model to **training and test data** in a single line of code

```
rf_fit <- last_fit(
     rf_wkfl, df_split)
```

❻ Evaluate model

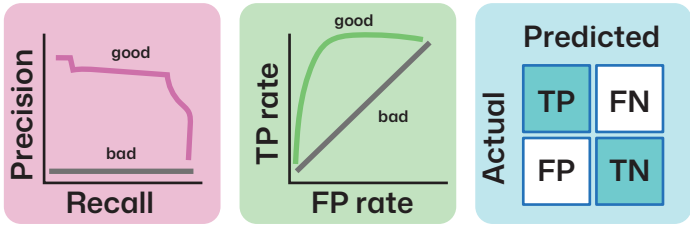




Collect test set performance to get **accuracy** and **AUROC**

Visualising model performance

PR curve | ROC curve | Confusion matrix



Precision-Recall (PR) curves show the trade-off between precision (sensitivity) and recall (specificity) to visualise how balanced a dataset is

ROC curves show the true positive rate (sensitivity) vs. false positive rate (1 - specificity) to evaluate a binary classifier.

The **confusion matrix** is a table showing 'truth' and 'predictions'

```
rf_fit %>%
     collect_predictions() %>%
     pr_curve(...) %>%
     roc_curve(...) %>%
     conf_mat(...) %>%
     autoplot()
```

one of

The **variable importance plot** tells us which variable is most important:

```
rf_fit %>%
     extract_fit_parsnip() %>%
     vip()
```